

CSYRUS TECHNOLOGIES

Engineering Internship

Technical Assessment

Time Limit
24 Hours

Expected Effort
6 – 10 Hours

What We're Looking For

We bring interns in as engineers from day one. We're not expecting perfect code — we're expecting an engineering mindset. Specifically, we want to see:

- Systems designed with clarity and purpose
- Code that others can read and maintain
- Delivery under real constraints
- Transparent decision-making
- A sense of ownership over the work
- Adherence to engineering best practices
- Thinking about scale from the start

Technology Stack

You must use exactly these technologies. Substitutions are not permitted.

Backend

Python 3.x

FastAPI

SQLAlchemy ORM

PostgreSQL

Frontend

React.js

Vite

React Router

Axios

Authentication — Google OAuth 2.0 Only

Email/password login is not permitted. The authentication flow must follow this exact sequence:

Login with Google → Google Consent Screen → Successful Auth → JWT Session Created → Protected App Access

What You're Building

Workflow Approval Management System

A platform where users submit approval requests and designated reviewers act on them. Think of it as a lightweight internal tool that mirrors how real businesses handle approvals — expense sign-offs, leave requests, content reviews, and so on.

User Roles

Requester → Log in via Google → Create, edit, view, and delete their own requests → Track current request status	Reviewer → View requests assigned to them → Approve or reject requests → Leave comments explaining their decision
---	--

Data Model

User

Field	Type / Values	Notes
id	UUID / Integer	Primary key
name	String	From Google profile
email	String	From Google profile
google_id	String	OAuth identifier
role	Requester Reviewer	User type
created_at	Timestamp	Auto-generated

Approval Request

Field	Type / Values	Notes
id	UUID / Integer	Primary key
title	String	Short description
description	Text	Full details
priority	LOW MEDIUM HIGH	Request urgency

status	PENDING APPROVED REJECTED	Current state
created_by	FK → User	Requester
reviewer_id	FK → User	Assigned reviewer
created_at / updated_at	Timestamps	Auto-managed

Review Action

Field	Type / Values	Notes
id	UUID / Integer	Primary key
request_id	FK → Approval Request	Associated request
action	APPROVED REJECTED	Decision taken
comments	Text	Reviewer notes
reviewed_by	FK → User	Reviewer
reviewed_at	Timestamp	Decision time

API Endpoints

Auth

```
GET /auth/google/login
GET /auth/google/callback
GET /auth/me
```

Requests (Requester)

```
POST /requests
GET /requests
GET /requests/{id}
PUT /requests/{id}
DELETE /requests/{id}
```

Reviewer

```
GET /reviewer/requests
POST /reviewer/requests/{id}/approve
POST /reviewer/requests/{id}/reject
```

Frontend Pages

Login

Google sign-in button and complete OAuth flow

Requester Dashboard

Profile summary, request statistics, and recent activity

Request Management

Create, edit, delete, and view request details. The request form captures: title, description, priority, and reviewer assignment.

Reviewer Dashboard

Pending requests list, individual request detail view, and approve/reject actions with comment input

Project Structure

Follow these structures closely. Deviating from them without a documented reason will count against you.

Backend

```
backend/
├── app/
│   ├── api/
│   ├── models/
│   ├── schemas/
│   ├── services/
│   ├── repositories/
│   ├── core/
│   └── database/
├── tests/
└── main.py
```

Frontend

```
src/
├── api/
├── components/
├── pages/
├── routes/
├── hooks/
├── services/
├── context/
├── utils/
└── tests/
```

Coding Standards

Backend — Mandatory

- Type hints on all functions
- Pydantic schemas for request/response validation
- Dependency injection for services and DB sessions
- Proper exception handling with meaningful HTTP status codes
- All secrets and config stored in environment variables
- Modular, layered architecture

Frontend — Mandatory

- Functional components throughout
- Reusable, composable component design
- Business logic separated from JSX
- API calls abstracted into a dedicated service/api layer
- No monolithic components — split early and split often

Testing

Testing is a major evaluation criterion — not optional.

Backend — pytest

- Auth: OAuth callback, JWT validation, protected route access
- Requests: create, update, delete, fetch
- Reviewer: approve, reject, status transitions, validation

Frontend — React Testing Library

- Components: Login, Dashboard, Request Form, Request List
- Flows: create request, approve request, reject request
- API layer: mocked calls and error handling

Git Workflow

Your commit history is part of the submission.

Branches

```
main
feature/auth
feature/backend
feature/frontend
feature/tests
```

Commit Message Format

```
feat(auth): implement Google OAuth login
feat(requests): add approval workflow
test(api): add reviewer endpoint tests
refactor(frontend): extract request card component
```

Avoid vague messages like: done, final, latest update, or anything that doesn't communicate intent.

Pull Requests: At least one PR is required. It must include a summary, a list of changes, and testing notes.

Documentation

README.md

- Project overview
- Setup instructions
- Environment variable reference
- How to run the app and tests
- API documentation

ENGINEERING_DECISIONS.md

- Architecture choices and rationale
- Auth design decisions
- Database design decisions
- Testing strategy
- Trade-offs accepted
- What you'd improve with more time

COLLABORATION.md

- Assumptions made
- Known limitations
- What would change in production
- Scalability considerations

Architecture Diagram

Show how the React frontend, FastAPI backend, PostgreSQL database, and Google OAuth service connect. Include the authentication flow. Accepted tools: Draw.io, Excalidraw, Mermaid, Lucidchart.

Deliverables

Submit all three of the following:

1	Backend GitHub Repository Source code, DB integration, Google OAuth, backend tests, and documentation
2	Frontend GitHub Repository React app, auth flow, all screens, frontend tests, and documentation
3	Demo Video (5–10 minutes) Via Loom, Google Drive, or YouTube (unlisted) — details below

What Your Demo Must Cover

- Google login flow, successful authentication, and protected access
- Creating, editing, deleting, and viewing a request
- Approving and rejecting a request with reviewer comments
- A walkthrough of your backend and frontend architecture
- Running pytest and npm test live on camera
- Your branch structure, commit history, and at least one PR

Scoring

Evaluation Area	Points
FastAPI Backend Design	20
React Frontend Architecture	15
Google OAuth Integration	10
Database Design	10
Unit Testing Quality	20
Code Quality & Standards	10
Git Workflow	5
Documentation	10
Total	100

Submission Checklist

Verify every item before you submit:

- ☐ Backend repository submitted
- ☐ Frontend repository submitted
- ☐ Demo video submitted
- ☐ Architecture diagram included
- ☐ README.md complete
- ☐ ENGINEERING_DECISIONS.md complete
- ☐ COLLABORATION.md complete
- ☐ All unit tests passing
- ☐ Meaningful commit history present
- ☐ At least one pull request created

A note on what "good" looks like

A candidate who completes every feature but ships messy, untested, undocumented code will score lower than one who implements fewer features with clean architecture, solid tests, and clear documentation.

We're not just evaluating what you built. We're evaluating how you think, how you work, and how you communicate — because that's what matters in a real engineering team.